

Emoticon Generation using Facial Emotion Recognition

Submitted in partial fulfillment of the requirements for
the award of
Bachelor of Engineering degree in Computer Science and Engineering
by

M SHWETA SINGHA (Reg. No. 37110712)

DEEPSHIKHA (Reg. No. 37110178)



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

SATHYABAMA

**INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)**

**Accredited with Grade "A" by NAAC
JEPPIAAR NAGAR, RAJIV GANDHI
SALAI, CHENNAI – 600 119**

MARCH - 2021



SATHYABAMA

INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)

Accredited with "A" grade by NAAC
Jeppiaar Nagar, Rajiv Gandhi Salai, Chennai – 600 119

www.sathyabama.ac.in



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this project report is the bonafide work of **M SHWETA SINGHA (Reg. No. 37110712)** and **DEEPSHIKHA (Reg. No. 37110178)** who carried out the project entitled "**Emoticon Generation using Facial Emotion Recognition** " under my supervision from **August 2020** to **March 2021**.

Internal Guide

Dr.A.C.Santha Sheela, M.E.,Ph.D.,

Head of the Department

Submitted for Viva voce Examination held on _____

Internal Examiner

External Examiner

DECLARATION

I **M SHWETA SINGHA/ DEEPSHIKHA** hereby declare that the Project Report entitled “**EMOTICON GENERATION USING FACIAL EMOTION RECOGNITION**” is done by me under the guidance of **Dr.A.C.Santha Sheela, M.E.,Ph.D.**, Department of Computer Science and Engineering at Sathyabama Institute of Science and Technology is submitted in partial fulfillment of the requirements for the award of Bachelor of Engineering degree in Computer Science and Engineering.

DATE:

PLACE: CHENNAI

SIGNATURE OF THE CANDIDATE

ACKNOWLEDGEMENT

I am pleased to acknowledge my sincere thanks to **the Board of Management of SATHYABAMA** for their kind encouragement in doing this project and for completing it successfully. I am grateful to them.

I convey my thanks to **Dr. T. Sasikala, M.E., Ph.D., Dean**, School of Computing, **Dr. S. Vigneswari, M.E., Ph.D., and Dr. L. Lakshmanan, M.E., Ph.D., Heads of the Department** of Computer Science and Engineering for providing me necessary support and details at the right time during the progressive reviews.

I would like to express my sincere and deep sense of gratitude to my Project Guide **Dr.A.C.Santha Sheela, M.E.,Ph.D., Assistant Professor**, for her valuable guidance, suggestions and constant encouragement paved the way for the successful completion of my project work.

I wish to express my thanks to all Teaching and Non-teaching staff members of the **Department of Computer Science and Engineering** who were helpful in many ways for the completion of the project.

ABSTRACT

Facial emoji generation is a human-computer interaction system. Face expressions are a key feature of non-verbal communication, and they play an important role in Human Computer Interaction. Emoji Generation in real time by recognizing the facial expression of a person has always been challenging. Facial expressions are vital to social communication between humans. As the world is emerging with new technology every day, there are more virtual interactions like text messages than the real ones. Emoticons help in social interaction virtually, with less exchange of words. This paper presents an approach of Emoji Generation using Facial Expression Recognition(FER) using Convolutional Neural Networks(CNN) with Machine Learning and Deep learning. This model created using CNN can be used to detect facial expressions in real time. The system can be used for analysis of emotions while users watch movie trailers or video lectures, feedback processing.

TABLE OF CONTENTS

ABSTRACT

v

LIST OF FIGURES

viii

CHAPTER No.	TITLE	PAGE No.
1.	INTRODUCTION	1
	1.1 FACIAL EMOTION RECOGNITION	2
	1.2 RESEARCH AND SIGNIFICANCE	2
2.	AIM AND SCOPE	4
	2.1 AIM OF PROJECT	4
	2.2 OBJECTIVES	4
	2.2 SCOPE	5
3.	SYSTEM DESIGN & METHODOLOGY	6
	3.1 EXISTING SYSTEM	6
	3.2 SYSTEM FUNCTIONALITIES	7
	3.3 LIBRARY AND PACKAGES	9
	3.4 REQUIREMENT SPECIFICATION	12
4.	SOFTWARE DEVELOPMENT METHODOLOGY	13
	4.1 DESCRIPTION OF DIAGRAM	13
	4.2 SYSTEM DESIGN	14
	4.3 SYSTEM FLOWCHART	15
	4.4 FACIAL EMOTION RECOGNITION USING CNN METHODOLOGY	16
	4.4.1 DATASET	16
	4.4.2 PROCESS OF FACIAL EMOTION RECOGNITION	17
	4.4.3 PREPROCESSING	17
	4.4.4 FACE DETECTION	18
	4.4.5 EMOTION CLASSIFICATION	18
	4.5 ARCHITECTURAL DESIGN	20

4.6	CNN METHODOLOGY	23
4.6.1	<i>DATA PREPROCESSING</i>	23
4.6.2	<i>IMPORTING LIBRARIES AND PACKAGES</i>	23
4.6.3	<i>INITIALIZE TRAINING AND VALIDATION GENERATOR</i>	24
4.6.4	<i>BUILDING THE MODEL USING CNN</i>	24
4.6.5	<i>COMPILE AND TRAIN THE MODEL</i>	26
4.6.6	<i>CALLBACK FUNCTIONS</i>	27
4.6.7	<i>USING OPENCV HAARCASCADE</i>	28
4.6.8	<i>TESTING THE MODEL</i>	29
4.7	APPLICATIONS	29
6.	RESULTS AND DISCUSSION	30
7.	CONCLUSION AND FUTURE WORK	36
	REFERENCES	37
	APPENDIX	
A.	PLAGIARISM REPORT	38
B.	PAPER SUBMISSION MAIL	39
C.	JOURNAL PAPER	40
D.	SOURCE CODE	41

LIST OF FIGURES

FIGURE No.	FIGURE NAME	PAGE No.
2.1	EXPECTED OUTPUT	5
3.1	CLASSIFICATION OF SEVEN BASIC EMOTIONS	6
4.1	BLOCK DIAGRAM	13
4.2	SYSTEM ARCHITECTURE	14
4.3	(A) FLOWCHART OF TRAINING PHASE	15
	(B) FLOWCHART OF TESTING PHASE	15
	(C) SCHEMA OF FER	15
4.4	(A) TRAINING, TESTING AND VALIDATION	16
4.4	(B) SAMPLE IMAGES	16
4.5	PROCESS FLOW	17
4.6	HAAR FEATURES	18
4.7	(A) ARCHITECTURE DIAGRAM OF EMOJI GENERATION	20
	(B) ARCHITECTURE DIAGRAM OF CNN	
4.8	IMPORTED LIBRARIES AND PACKAGES	24
4.9	TRAINING AND VALIDATION GENERATOR	25
4.10	(A) TRIAL 1	25
	(B) TRIAL 2	25
4.11	DETECTED FACIAL EMOTIONS	28
5.1	EMOJI GENERATED	31
	(I) NEUTRAL	31
	(II) FEARFUL	31
	(III) SURPRISED	32
	(IV) DISGUST	32
	(V) HAPPY	32
	(VI) ANGRY	32
	(VII) SAD	32
5.2	CONFUSION MATRIX	35

CHAPTER 1

INTRODUCTION

1. INTRODUCTION

Emoji are ideograms and smileys used in electronic messages and web pages. Emoji exist in various genres, including facial expressions, common objects, places and types of weather, and animals. They are much like emoticons, but emoji are pictures rather than typographic approximations; the term "emoji" in the strict sense refers to such pictures which can be represented as encoded characters, but it is sometimes applied to messaging stickers by extension. Originally meaning pictograph, the word *emoji* comes from Japanese *e* ("picture") + *moji* ("character"); the resemblance to the English words *emotion* and *emoticon* is purely coincidental. Originating on Japanese mobile phones in 1997, emoji became increasingly popular worldwide in the 2010s after being added to several mobile operating systems.

Emojis are essential to communicate emotion, something that words cannot portray. However, they do not hold value in the academic world or in a context that demands an objective voice. Emojis are meant to be fun, light-hearted, and convey a broad range of emotions efficiently and in a way that words sometimes cannot. Emoji's or avatars are ways to indicate nonverbal cues. These cues have become an essential part of online chatting, product review, brand emotion, and many more. It also leads to increasing data science research dedicated to emoji-driven storytelling.

A facial expression is one or more motions or positions of the muscles beneath the skin of the face. According to one set of controversial theories, these movements convey the emotional state of an individual to observers. Facial expressions are a form of nonverbal communication. Facial expressions are vital to social communication between humans. Facial expression classifiers generalize the learned features to recognize different expressions from unseen faces. With advancements in computer vision and deep learning, it is now possible to detect human emotions from images in an improved way. A Convolutional Neural Network (CNN) is a Deep Learning Algorithm which takes an image as the input,

assigning importance to various objects in the image so that it can differentiate it from others. The preprocessing required in a CNN model is lower as compared to other classification models. We are using CNN because it has an ability which automatically detects the important features without any human supervision.

1.1 FACIAL EMOTION RECOGNITION

Facial emotion recognition is the process of detecting human emotions from facial expressions. The human brain recognizes emotions automatically, and software has now been developed that can recognize emotions as well. This technology is becoming more accurate all the time, and will eventually be able to read emotions as well as our brains do. AI can detect emotions by learning what each facial expression means and applying that knowledge to the new information presented to it. Emotional artificial intelligence, or emotion AI, is a technology that is capable of reading, imitating, interpreting, and responding to human facial expressions and emotions. One of the important ways humans display emotions is through facial expressions. Facial expression recognition is one of the most powerful, natural and immediate means for human beings to communicate their emotions and intentions. Humans can be in some circumstances restricted from showing their emotions, such as hospitalized patients, or due to deficiencies; hence, better recognition of other human emotions will lead to effective communication. Automatic human emotion recognition has received much attention recently with the introduction of IOT and smart environments at hospitals, smart homes and smart cities. Intelligent personal assistants (IPAs), such as Siri, Alexia, Cortana and others, use natural language processing to communicate with humans, but when augmented with emotions, it increases the level of effective communication and human-level intelligence.

1.2 RESEARCH AND SIGNIFICANCE

Human facial expressions can be easily classified into 7 basic emotions: happy, sad, surprise, fear, anger, disgust, and neutral. Our facial emotions are expressed through activation of specific sets of facial muscles. These sometimes subtle, yet complex, signals in an expression often contain an abundant amount of information about our state of mind. Through facial emotion recognition, we are

able to measure the effects that content and services have on the audience/users through an easy and low-cost procedure. For example, retailers may use these metrics to evaluate customer interest. Healthcare providers can provide better service by using additional information about patients' emotional state during treatment. Entertainment producers can monitor audience engagement in events to consistently create desired content. Humans are well-trained in reading the emotions of others, in fact, at just 14 months old, babies can already tell the difference between happy and sad. But can computers do a better job than us in accessing emotional states? To answer the question, We designed a deep learning neural network that gives machines the ability to make inferences about our emotional states. In other words, we give them eyes to see what we can see.

In this deep learning project, we aim to classify human facial expressions in an effective manner to filter and map corresponding emoji or avatars. The application can be used in large corporations or firms for collecting feedback from the customers in real time. The results generated from the application can be further used in research and development processes. Emoji are used more and more frequently in network communication, and the way they are used is becoming more and more diversified as well. They not only have unique semantic and emotional features, but are also closely related to marketing, law, health care and many other areas. The research on emoji has become a hot topic in the academic field, and more and more scholars from the fields of computing, communication, marketing, behavioral science and so on are studying them. This paper reviews the developmental history and usage of emoji, details the emotional and linguistic features of emoji, summarizes the results of research on emoji in different fields, and puts forward future research directions.

CHAPTER 2

AIM AND SCOPE

2.1 AIM OF PROJECT

The objective of this project is to develop Automatic Facial Emoji Generation System which can take human facial images containing some expression as input and recognize and classify it and generate emoji based on seven different expression class such as :

1. Angry
2. Sad
3. Happy
4. Disgust
5. Neutral
6. Surprised
7. Fear

The facial expression of human emotion is one of the major topics in facial recognition, and it can generate both technical and everyday application beyond laboratory experiment. This projection constructs a system of deep learning models to classify a given image of human facial emotion into one of the seven basic human emotions. Then we will map the classified emotion to an emoji or an avatar.

2.2 OBJECTIVES

In this deep learning project, we have built a convolution neural network to recognize facial emotions. We have trained our model on the FER2013 dataset. Then we are mapping those emotions with the corresponding emojis or avatars. We are using OpenCV's haar cascade xml we are getting the bounding box of the faces in the webcam. Then we feed these boxes to the trained model for classification.

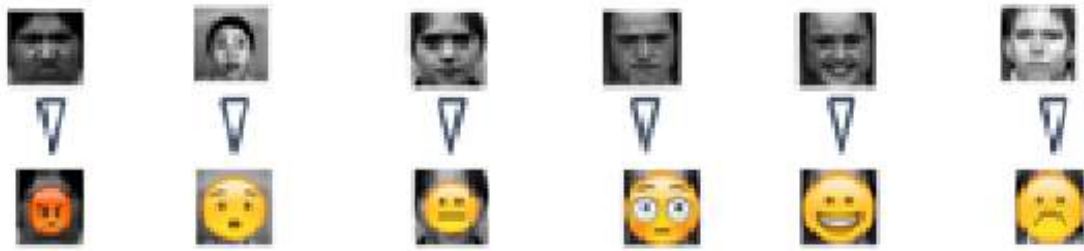


FIG 2.1: EXPECTED OUTPUT

The emoji are correctly imposed on their corresponding faces. The input will be raw image of the expression, and output will be shown as above

2.3 SCOPE

The primary scope of this project is to establish a model that can classify seven basic emotions: happy, sad, surprise, angry, disgust, neutral, and fear and to achieve the accuracy better than the baseline. In addition to this, our project also aims to analyze the results of our model in terms of accuracy for each class. In the future, the model is expected to perform wild emotion recognition that has more complex variance of condition than lab condition images.

The use of Emoji in marketing activities can enhance the appeal of these activities and bring them closer to the younger generation. It can also have an impact on consumers, including optimizing consumer experience, improving purchase intention, and changing perceptions of brands.Emoji can be used to measure users' emotions and depict the portraits of users.

We have also been motivated observing the benefits of physically handicapped people like deaf and dumb. But if any normal human being or an automated system can understand their needs by observing their facial expression then it becomes a lot easier for them to make the fellow human or automated system understand their needs. Significant debate has risen in the past regarding the emotions portrayed in the world famous masterpiece of Mona Lisa. British Weekly „New Scientist“ has stated that she is in fact a blend of many different emotions, 83%happy, 9% disgusted, 6% fearful, 2% angry.

CHAPTER 3

SYSTEM DESIGN & METHODOLOGY

3.1 EXISTING SYSTEM

The study of nonverbal communication via emotions originated with Darwin's claim that emotion expressions evolved in humans from pre-human nonverbal displays. Furthermore, according to Ekman, there are seven basic emotions which have acquired a special status among the scientific community: Anger, Disgust, Fear, Happiness, Neutral, Sadness, and Surprise. The compactness of emojis reduces the effort of input to express not only emotions, but also serves to adjust message tone, increase message engagement, manage conversations and maintain social relationships. Moreover, emojis do not have language barriers, making it possible for users across countries and cultural backgrounds to communicate. In a study by Barbieri et al., they found that the overall semantics of the subset of the emojis they studied is preserved across US English, UK English, Spanish, and Italian. As validation of the usefulness of mapping emojis to emotions, preliminary investigations reported by Jaeger et al. suggest that emoji may have potential as a method for direct measurement of emotional associations to foods and beverages.



FIG 3.1 CLASSIFICATION OF SEVEN BASIC EMOTION

Facial Emotion Recognition has become an increasingly researched topic in recent years, mainly because it has a lot of applications in the fields of Computer Vision, robotics, and Human Computer Interaction. In a study on the facial recognition technology (FERET) dataset, Paul Ekman has presented 7 universal expressions (anger, fearful, happy, sad, neutral, disgust, and surprise) with the

positioning of faces, and the muscular movements required to create these expressions in his study (Ekman, 1997). The Facial Action Coding System (FACS), developed by Swedish anatomist Carl-Herman Hjortsjö, is a coding system used to taxonomise human facial movements based on their appearance on the face. This system, which was later adopted by Ekman & Friesen (2003), is also a useful method of classifying human expressions. FER systems were mostly implemented using the FACS in the past. However, recently there has been a trend to implement FER using classification algorithms such as SVM, neural networks, and the Fisherface algorithm (Alshamsi, Kepuska & Meng, 2017; Fathallah, Abdi & Douik, 2017; Lyons, Budynek & Akamatsu, 1999).

There are several datasets available for research in the field of Facial Expression Recognition, such as the Japanese Female Facial Expressions (JAFFE), Extended Cohn Kanade dataset (CK+), and the FER2013 dataset (Canade, Cohn & Tian, 2000; Lucey et al., 2010; Goodfellow et al., 2013). The type and number of images, the method of labelling the images varies in each dataset. The CK+ dataset uses the FACS system for labelling faces and contains the Action Units (AU's) for each facial image.

There are several challenges with implementing the FER system. Most datasets consist of images of posed people with a certain expression. This is the first challenge; as real time applications require a model with expressions which are not posed or directed. The second challenge is that the labels in the datasets are broadly classified, which means that in real time there might be some expressions which the system might be able to classify correctly. Many FER systems, such as Affectiva, and Microsoft's Emotion API (McDuff et al., 2016; Linn, 2015). These systems have become very popular in applications where FER is required.

3.2 SYSTEM FUNCTIONALITIES

Deep learning is currently a hot topic in Machine Learning and is also an essential tool in the field of Artificial Intelligence. Deep Learning is good at doing image recognition, object detection, natural language translation, trend prediction. Emoji is a visual symbol widely used in wireless communication. It includes faces, hand gestures, animals, human figures, and signs. Instead of typographic, emoji

are actual pictures. As there are increasing uses of emojis, the needs for an instant generator of emojis have become urgent. With the promising results of image recognition tasks by some deep learning models, we propose a real-time emoji generator, which enables the user to generate an emoji given a corresponding human facial expression. In this work, we applied deep learning models to perform feature extraction and pre-processed our data with data augmentation strategies and morphology operations. Our model uses the model of the convolutional neural network. It consists of convolutional layers, ReLu layers, and max-pooling layers. For our model, we have two sets of convolutional layers, ReLu layers, and max-pooling layers. After that, we put a fully-connected layer, a ReLu layer, and a softmax layer to predict the label.

Facial expression recognition is a process performed by humans or computers, which consists of:

1. Locating faces in the scene (e.g., in an image; this step is also referred to as face detection),
2. Extracting facial features from the detected face region (e.g., detecting the shape of facial components or describing the texture of the skin in a facial area; this step is referred to as facial feature extraction),
3. Analyzing the motion of facial features and/or the changes in the appearance of facial features and classifying this information into some facial-expression interpretation categories such as facial muscle activations like smile or frown, emotion (affect)categories like happiness or anger, attitude categories like (dis)liking or ambivalence, etc.(this step is also referred to as facial expression interpretation).
4. Converting the detected facial expression into emojis by mapping the predicted results.

Several Projects have already been done in this field and our goal will not only be to develop an Automatic Facial Expression Emoji Generation System but also improving the accuracy of this system compared to the other available system.

3.3 LIBRARY AND PACKAGES

- **OpenCV:** OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in commercial products. Being a BSD-licensed product, OpenCV makes it easy for businesses to utilize and modify the code. The library has more than 2500 optimized algorithms, which includes a comprehensive set of both classic and state-of-the-art computer vision and machine learning algorithms. These algorithms can be used to detect and recognize faces, identify objects, classify human actions in videos, track camera movements, track moving objects, extract 3D models of objects, produce 3D point clouds from stereo cameras, stitch images together to produce a high resolution image of an entire scene, find similar images from an image database, remove red eyes from images taken using flash, follow eye movements, recognize scenery and establish markers to overlay it with augmented reality, etc. OpenCV has more than 47 thousand people in the user community and an estimated number of downloads exceeding 14 million. The library is used extensively in companies, research groups and by governmental bodies. It has C++, Python, Java and MATLAB interfaces and supports Windows, Linux, Android and Mac OS. OpenCV leans mostly towards real-time vision applications and takes advantage of MMX and SSE instructions when available. A full-featured CUDA and OpenCL interface is being actively developed right now. There are over 500 algorithms and about 10 times as many functions that compose or support those algorithms. OpenCV is written natively in C++ and has a templated interface that works seamlessly with STL containers. OpenCV's application areas include :

- 2D and 3D feature toolkits
- Facial recognition system
- Gesture recognition
- Human-computer interaction (HCI)
- Object identification
- Stereopsis stereo vision: Depth perception from 2 cameras
- Motion tracking
- Augmented reality

To support some of the above areas, OpenCV includes a statistical machine learning library that contains :

- Decision tree learning
 - k-nearest neighbor algorithm
 - Naive Bayes classifier
 - Artificial neural networks
 - Random forest Random forest
 - Support vector machine (SVM)
 - Deep neural networks (DNN)
- **Numpy** : NumPy is an acronym for "Numeric Python" or "Numerical Python". It is an open source extension module for Python, which provides fast precompiled functions for mathematical and numerical routines. Furthermore, NumPy enriches the programming language Python with powerful data structures for efficient computation of multi-dimensional arrays and matrices. The implementation is even aiming at huge matrices and arrays. Besides that the module supplies a large library of high-level mathematical functions to operate on these matrices and arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
 - Sophisticated (broadcasting) functions
 - Tools for integrating C/C++ and Fortran code
 - Useful linear algebra, Fourier Transform, and random number capabilities
- **Keras**: Keras is a high-level neural networks API, written in Python and capable of running on top of TensorFlow, CNTK, or Theano. It was developed with a focus on enabling fast experimentation.

Keras contains numerous implementations of commonly used neural network building blocks such as layers, objectives, activation functions, optimizers, and a host of tools to make working with image and text data easier. The code is hosted on GitHub, and community support forums include the GitHub issues page, and a Slack channel.

Keras allows users to productize deep models on smartphones (iOS and

Android), on the web, or on the Java Virtual Machine. It also allows use of distributed training of deep learning models on clusters of Graphics Processing Units (GPU).

- **SciPy** : SciPy (Scientific Python) is often mentioned in the same breath with NumPy. SciPy extends the capabilities of NumPy with further useful functions for minimization, regression, Fourier-transformation and many others. NumPy is based on two earlier Python modules dealing with arrays. One of these is Numeric. Numeric is like NumPy, a Python module for high-performance, numeric computing, but it is obsolete nowadays. Another predecessor of NumPy is Numarray, which is a complete rewrite of Numeric but is deprecated as well. NumPy is a merger of those two, i.e. it is built on the code of Numeric and the features of Numarray.
- **TensorFlow** : TensorFlow is a Python library for fast numerical computing created and released by Google. It is a foundation library that can be used to create Deep Learning models directly or by using wrapper libraries that simplify the process built on top of TensorFlow.
- **Haar Cascade Classifier in OpenCv** : Haar feature-based cascade classifiers is an effectual machine learning based approach, in which a cascade function is trained using a sample that contains a lot of positive and negative images. The outcome of AdaBoost classifiers is that the strong classifiers are divided into stages to form cascade classifiers. The term “cascade” means that the classifier thus produced consists of a set of simpler classifiers which are applied to the region of interest until the selected object is discarded or passed. The cascade classifier splits the classification work into two stages: training and detection. The training stage does the work of gathering the samples which can be classified as positive and negative. The cascade classifier employs some supporting functions to generate a training dataset and to evaluate the prominence of classifiers. In order to train the cascade classifier, we need a set of positive and negative samples.

3.4 REQUIREMENT SPECIFICATION

This proposed software runs effectively on a computing system that has the minimum requirements. The requirements are split into three categories, namely:

Software Requirements

The basic software requirements to run the program are:

- i. Microsoft Windows XP, Windows 7, Windows 10
- ii. Python
- iii. IDE e.g. Google Colab, Anaconda Navigator, Jupyter.
- iv. Browser e.g. Mozilla Firefox, Safari, chrome, etc.
- v. TensorFlow, openCV.

Data set Requirements

The dataset required for training the Model is:

- i. FER2013 from kaggle

Hardware Requirements

The basic hardware required to run the program are:

- i. Hard disk of 500 GB or higher.
- ii. System memory (RAM) of 4GB or higher.
- iii. I3 processor-based computer or higher.
- iv. Web Camera.

Software Interfaces

1. Microsoft Word 2007
2. Dataset Storage : Microsoft Excel
3. Operating System : Windows10

CHAPTER 4

SOFTWARE DEVELOPMENT METHODOLOGY

4.1 DESCRIPTION OF DIAGRAM

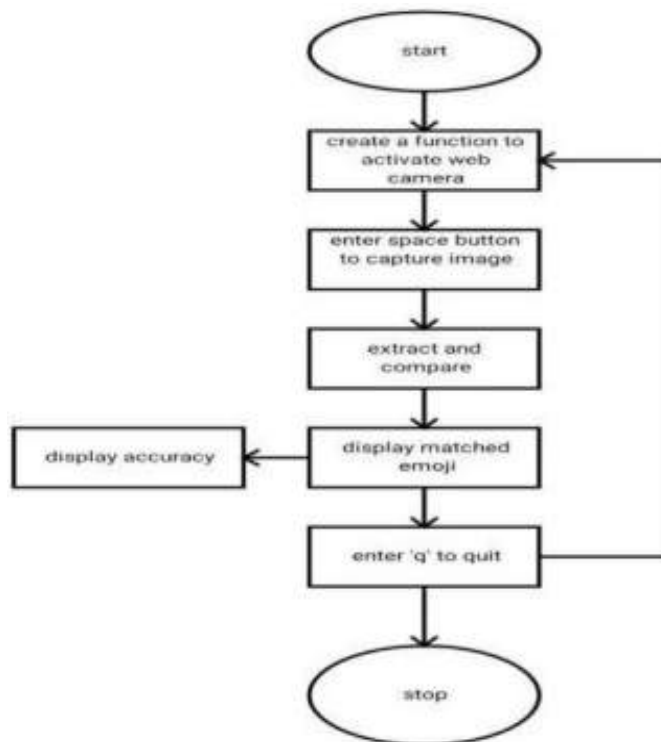


Fig 4.1: BLOCK DIAGRAM

In the Fig 4.1, Facial expressions can be described as the arrangement of facial muscles to convey a certain emotional state to the observer in simple words. Emotions can be divided into six broad categories—Anger, Disgust, Fear, Happy, Sad, Surprise, and Neutral. In this, train a model to differentiate between these, train a convolutional neural network using the FER2013 dataset and will use various hyper-parameters to fine-tune the model.

The design starts with the initializing CNN model by taking an input image (static or dynamic) by adding a convolution layer, pooling layer, flatten layers, and dense layers. Convolution layers will be added for better accuracy for large datasets. The dataset is collected from a CSV file (in pixel format) and it's converted into images and then classify emotions with respective expressions.

4.2 SYSTEM DESIGN

System design shows the overall design of the system. In this section we discuss in detail the design aspects of the system :

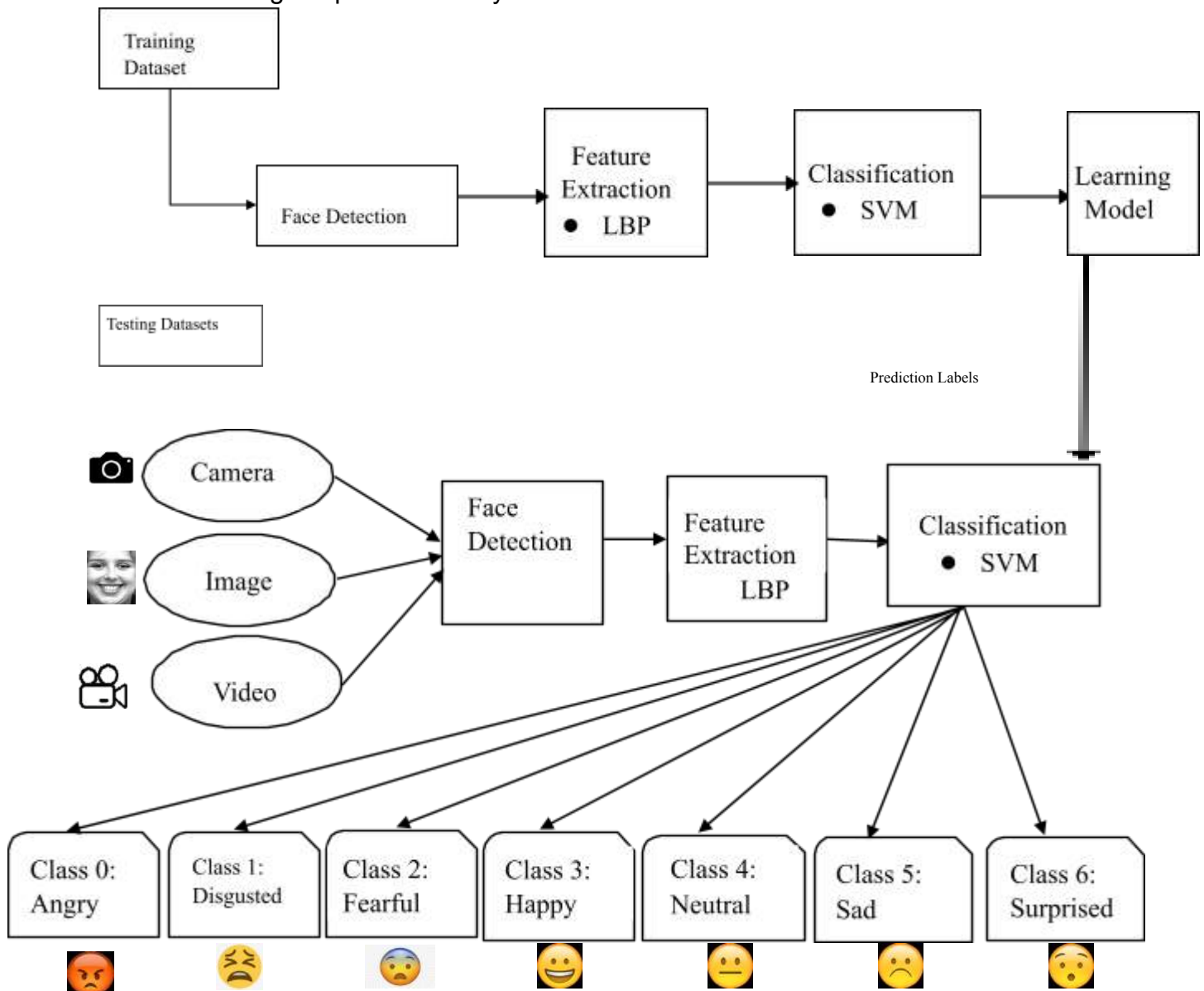


Fig 4.2 SYSTEM ARCHITECTURE

Here emotions are classified as happy, sad, angry, surprise, neutral, disgust, and fear with 34,488 images for the training dataset and 1,250 for testing. Each emotion is expressed with different facial features like eyebrows, opening the mouth, Raised cheeks, wrinkles around the nose, wide-open eyelids and many others. Trained the large dataset for better accuracy and result that is the object class for an input image. Based on those features it performs convolution layers and max pooling. These are the seven different universal emotions with the following expressions above Fig 4.2.

4.3 SYSTEM FLOWCHART

The facial expression recognition system is implemented using convolutional neural networks. The block diagram of the system is shown in following figures:

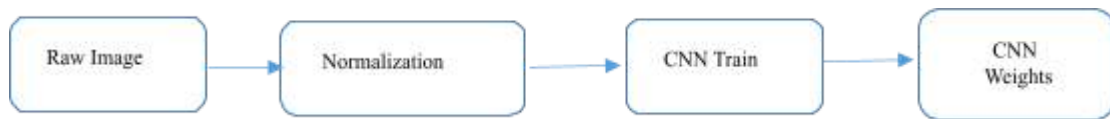


Fig 4.3(a) Flowchart of Training Phase

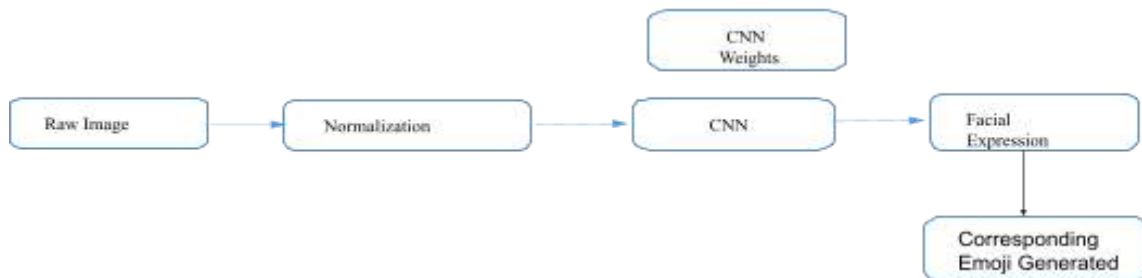


Fig 4.3(b) Flowchart of Testing Phase

During training, the system receives training data comprising grayscale images of faces with their respective expression label and learns a set of weights for the network. The training step took as input an image with a face. Thereafter, an intensity normalization is applied to the image. The normalized images are used to train the Convolutional Network. To ensure that the training performance is not affected by the order of presentation of the examples, validation dataset is used to choose the final best set of weights out of a set of training exercises performed with samples presented in different orders. The output of the training step is a set of weights that achieve the best result with the training data. During the test, the system received a grayscale image of a face from the test dataset, and output the predicted expression by using the final network weights learned during training. Its output is a single number that represents one of the seven basic expressions.

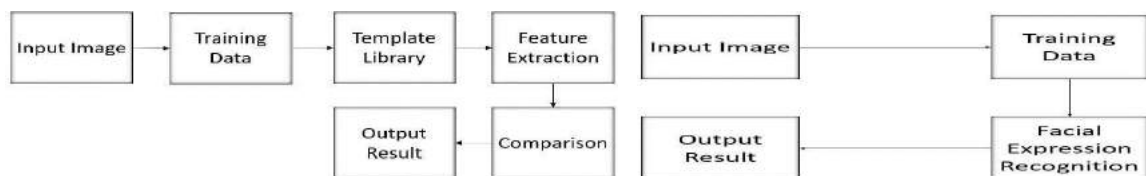


Fig 4.3(c) Schema of facial expression recognition

The original structure contains six steps which are input image, training data, template library, feature extraction, comparison and output result, as shown in Figure 4.3(c). However, a simplified structure that is used in this paper only has four steps after we

combine the step of template library, feature extraction and comparison to facial expression recognition, as shown in Figure above. It will greatly increase the efficiency and reduce the running time.

4.4 FACIAL EMOTION RECOGNITION USING CNN METHODOLOGY

4.4.1 Dataset

The dataset from a Kaggle Facial Expression Recognition Challenge (FER2013) is used for the training and testing. It comprises pre-cropped, 48-by-48-pixel grayscale images of faces each labeled with one of the 7 emotion classes: anger, disgust, fear, happiness, sadness, surprise, and neutral. Dataset has a training set of 35887 facial images with facial expression labels.. The dataset has class imbalance issues, since some classes have a large number of examples while some have few. The dataset is balanced using oversampling, by increasing numbers in minority classes. The balanced dataset contains 40263 images, from which 29263 images are used for training, 6000 images are used for testing, and 5000 images are used for validation.

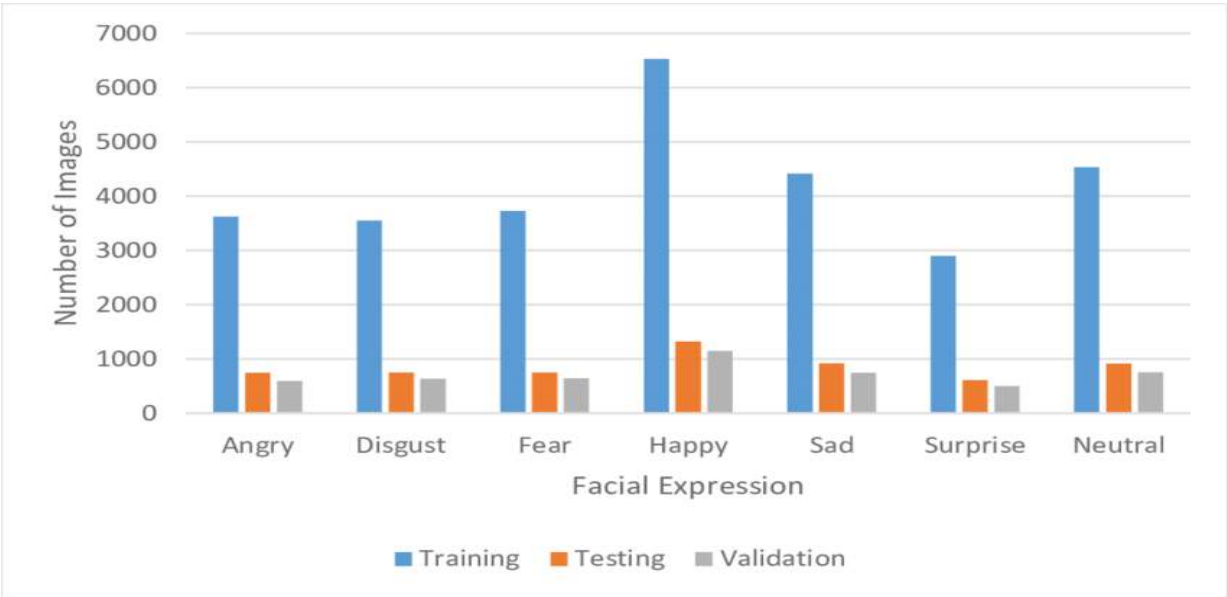


Fig 4.4(a): Training, Testing and Validation Data distribution

The images in the FER2013 dataset have size 48x48 and are black and white images. The FER2013 dataset contains images that vary in viewpoint, lighting, and scale. Fig.4.4(b) shows some sample images from the FER2013 dataset, and Table 4.4(c) illustrates the description of the dataset.



Fig. 4.4(b). Sample images from the FER2013 dataset

Table 4.1 Description of the FER2013 dataset

Label	Number of images	Emotion
0	4593	Angry
1	547	Disgust
2	5121	Fear
3	8989	Happy
4	6077	Sad
5	4002	Surprise
6	6198	Neutral

4.4.2 Process of Facial Expression Recognition

The process of FER has three stages. The preprocessing stage consists of preparing the dataset into a form which will work on a generalized algorithm and generate efficient results. In the face detection stage, the face is detected from the images that are captured real time. The emotion classification step consists of implementing the CNN algorithm to classify input images into one of seven classes.

These stages are described using in a flowchart in Fig. 2:

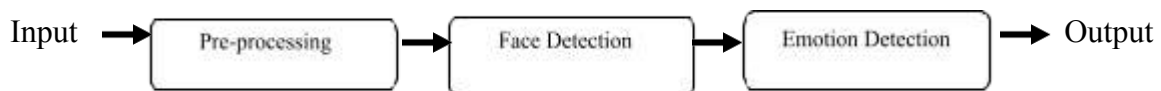


Fig.4.5 Process flow

4.4.3 Preprocessing

The input image to the FER may contain noise and have variation in illumination, size, and color. To get accurate and faster results on the algorithm, some preprocessing operations were done on the image. The preprocessing strategies used are conversion of image to grayscale, normalization, and resizing of image.

1. Normalization - Normalization of an image is done to remove illumination variations and obtain improved face image
2. Grayscale - Grayscale is the process of converting a colored image input into an image whose pixel value depends on the intensity of light on the image.

Grayscale is done as colored images are difficult to process by an algorithm.

3. Resizing - The image is resized to remove the unnecessary parts of the image. This reduces the memory required and increases computation speed.

4.4.4 Face Detection

Face detection is the primary step for any FER system. For face detection, Haar cascades were used (Viola & Jones, 2001). The Haar cascades, also known as the Viola Jones detectors, are classifiers which detect an object in an image or video for which they have been trained. They are trained over a set of positive and negative facial images. Haar cascades have proved to be an efficient means of object detection in images and provide high accuracy.

Haar features detect three dark regions on the face, for example the eyebrows. The computer is trained to detect two dark regions on the face, and their location is decided using fast pixel calculation. Haar cascades successfully remove the unrequired background data from the image and detect the facial region from the image.

The face detection process using the Haar cascade classifiers was implemented in OpenCV. This method was originally proposed by Papageorgiou et al, using rectangular features which are shown in figure 3 (Mohan, Papageorgiou & Poggio, 2001; Papageorgiou, Oren & Poggio, 1998).



Fig.4.6 Haar features

4.4.5 Emotion Classification

In this step, the system classifies the image into one of the seven universal expressions - Happiness, Sadness, Anger, Surprise, Disgust, Fear, and Neutral as labelled in the FER2013 dataset. The training was done using CNN, which is a category of neural networks proved to be productive in image processing. The dataset was first split into training and test datasets, and then it was trained on the training set. Feature extraction process was not done on the data before feeding it into CNN.

The approach followed was to experiment with different architectures on the CNN, to achieve better accuracy with the validation set, with minimum overfitting. The emotion classification step consists of the following phases:

- 1) Splitting of Data:

The dataset was split into 3 categories according to the “Usage” label in the FER2013 dataset: Training, PublicTest, and PrivateTest. The Training and PublicTest set were used for generation of a model, and the PrivateTest set was used for evaluating the model.

2) Training and Generation of model:

The neural network architecture consists of the following layers:

i. Convolution Layer:

In the convolution layer, a randomly instantiated learnable filter is slid, or convolved over the input. The operation performs the dot product between the filter and each local region of the input. The output is a 3D volume of multiple filters, also called the feature map.

ii. Max Pooling:

The pooling layer is used to reduce the spatial size of the input layer to lower the size of input and the computation cost.

iii. Fully connected layer:

In the fully connected layer, each neuron from the previous layer is connected to the output neurons. The size of the final output layer is equal to the number of classes in which the input image is to be classified.

iv. Activation function:

Activation functions are used to reduce the overfitting. In the CNN architecture, the ReLu activation function has been used. The advantage of the ReLu activation function is that its gradient is always equal to 1, which means that most of the error is passed back during back-propagation .

$$f(x) = \max(0, x)$$

Equation 1: Equation of ReLu Activation Function

v. Softmax:

The softmax function takes a vector of N real numbers and normalizes that vector into a range of values between (0, 1).

vi. Batch Normalization:

The batch normalizer speeds up the training process and applies a transformation that maintains the mean activation close to 0 and the activation standard deviation close to 1.

3) Evaluation of model:

The model generated during the training phase was then evaluated on the

validation set, which consisted of 3589 images.

4) Using model to classify real time images:

The concept of transfer learning can be used to detect emotion in images captured in real time. The model generated during the training process consists of pretrained weights and values, which can be used for implementation of a new facial expression detection problem. As the model generated already contains weights, FER becomes faster for real time images.

4.5 ARCHITECTURAL DESIGN

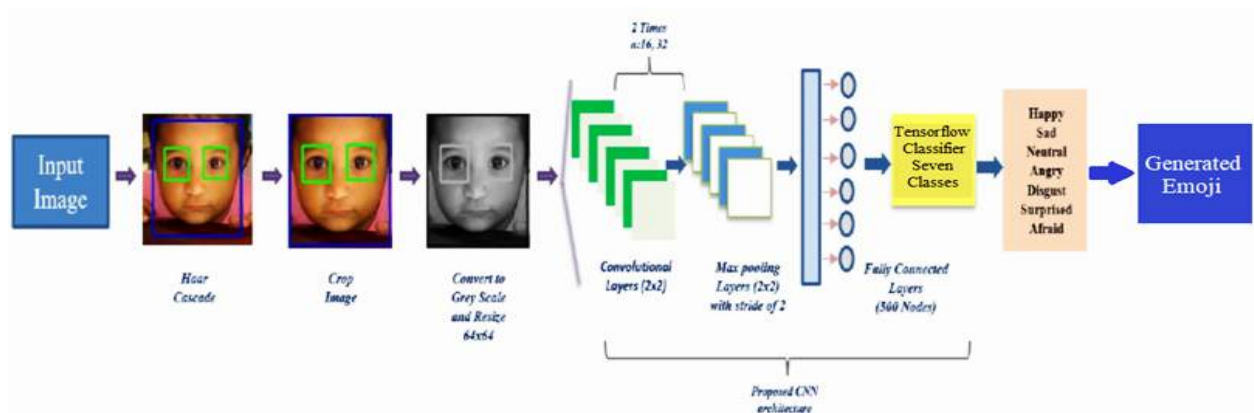


Fig 4.7(a) Architecture Diagram of Emoji Generation from FER

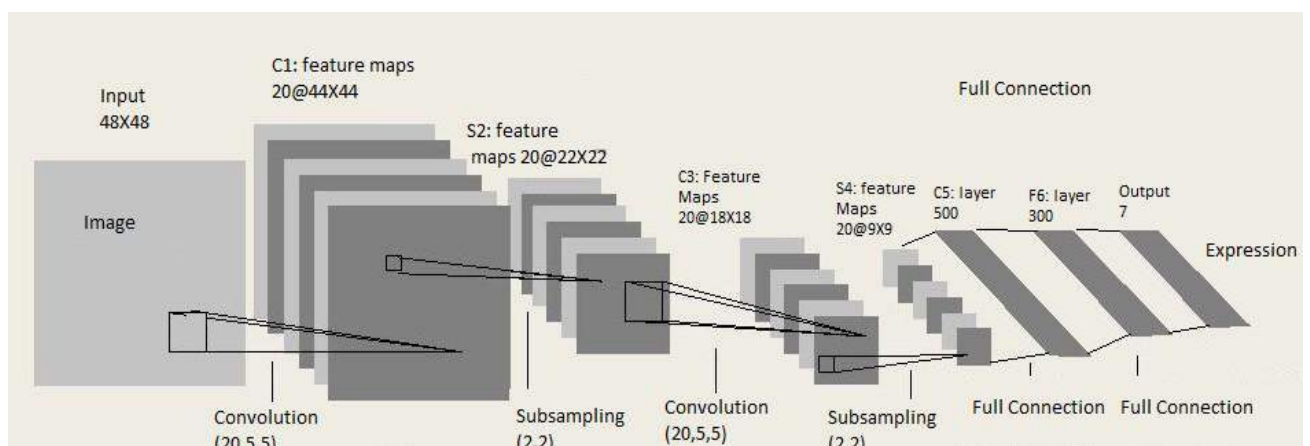


Fig 4.7(b): Architecture of CNN

A typical architecture of a convolutional neural network contains an input layer, some convolutional layers, some fully-connected layers, and an output layer. CNN is designed with some modification on LeNet Architecture. It has 6 layers without

considering input and output. The architecture of the Convolution Neural Network used in the project is shown in the above figure 4.5(b).

1. Input Layer:

The input layer has predetermined, fixed dimensions, so the image must be pre-processed before it can be fed into the layer. Normalized gray scale images of size 48 X 48 pixels from Kaggle dataset are used for training, validation and testing. For testing proposed laptop webcam images are also used, in which face is detected and cropped using OpenCV Haar Cascade Classifier and normalized.

2. Convolution and Pooling (ConvPool) Layers:

Convolution and pooling is done based on batch processing. Each batch has N images and CNN filter weights are updated on those batches. Each convolution layer takes image batch input of four dimension N x Color-Channel x width x height. Feature maps or filters for convolution are also four dimensional (Number of feature maps in, number of feature maps out, filter width, filter height). In each convolution layer, four dimensional convolution is calculated between image batch and feature maps. After convolution only the parameter that changes is image width and height.

$$\begin{aligned}\text{New image width} &= \text{old image width} - \text{filter width} + 1 \\ \text{New image height} &= \text{old image height} - \text{filter height} + 1\end{aligned}$$

After each convolution layer downsampling / subsampling is done for dimensionality reduction. This process is called Pooling. Max pooling and Average Pooling are two famous pooling methods. In this project max pooling is done after convolution. Pool size of (2 x 2) is taken, which splits the image into a grid of blocks each of size (2 x 2) and takes a maximum of 4 pixels. After pooling only height and width are affected.

Two convolution layers and a pooling layer are used in the architecture. The first convolution layer size of the input image batch is (N x 1 x 48 x 48). Here, the size of the image batch is N, number of color channels is 1 and both image height and width are 48 pixels. Convolution with a feature map of (1 x 20 x 5 x 5) results in an image batch of size (N x 20 x 44 x 44). After convolution pooling is

done with a pool size of (2×2) , which results in an image batch of size $(N \times 20 \times 22 \times 22)$. This is followed by a second convolution layer with a feature map of $20 \times 20 \times 5 \times 5$, which results in an image batch of size $(N \times 20 \times 18 \times 18)$. This is followed by a pooling layer with pool size (2×2) , which results in an image batch of size $(N \times 20 \times 9 \times 9)$.

3. Fully Connected Layer

This layer is inspired by the way neurons transmit signals through the brain. It takes a large number of input features and transforms features through layers connected with trainable weights. Two hidden layers of size 500 and 300 units are used in fully-connected layers. The weights of these layers are trained by forward propagation of training data then backward propagation of its errors. Back propagation starts from evaluating the difference between prediction and true value, and back calculates the weight adjustment needed to every layer before. We can control the training speed and the complexity of the architecture by tuning the hyper-parameters, such as learning rate and network density. Hyper-parameters for this layer include learning rate, momentum, regularization parameter, and decay.

The output from the second pooling layer is of size $N \times 20 \times 9 \times 9$ and input of the first hidden layer of fully-connected layer is of size $N \times 500$. So, the output of the pooling layer is flattened to $N \times 1620$ size and fed to the first hidden layer. Output from the first hidden layer is fed to the second hidden layer. Second hidden layer is of size $N \times 300$ and its output is fed to the output layer of size equal to the number of facial expression classes.

4. Output Layer

Output from the second hidden layer is connected to the output layer having seven distinct classes. Using Softmax activation function, output is obtained using the probabilities for each of the seven classes. The class with the highest probability is the predicted class.

4.6 CNN METHODOLOGY

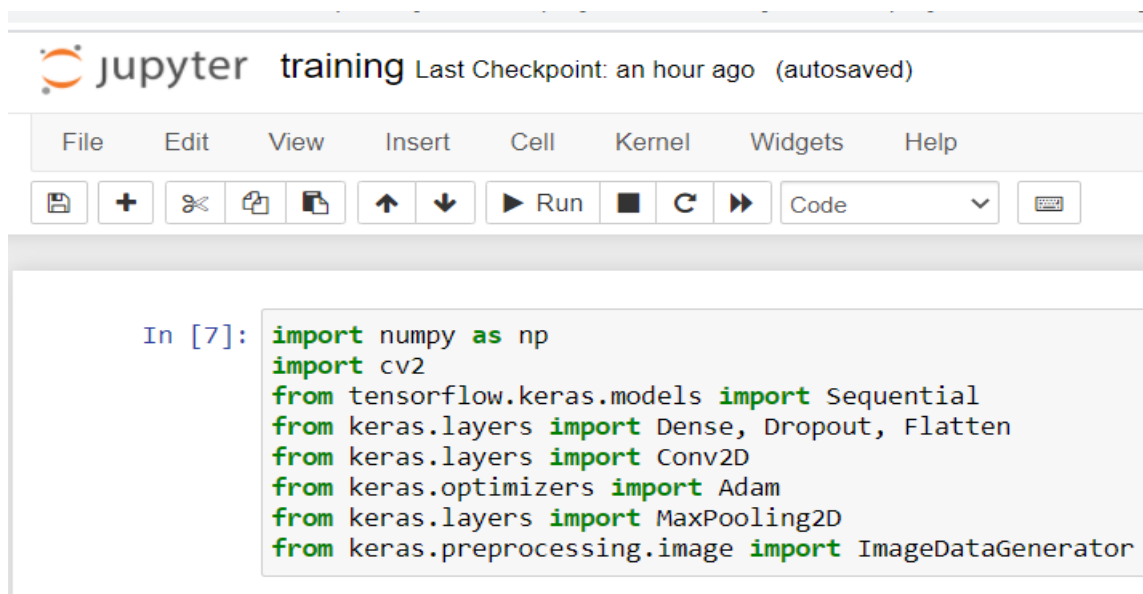
CNN architecture for facial expression recognition as mentioned above was

implemented in Python. Along with Python programming language, Numpy, Keras, CV2 libraries are used. In the below steps will build a convolution neural network architecture and train the model on FER2013 dataset for Emotion recognition from images.

4.6.1 Data Preprocessing

The dataset consists of 48*48 pixel gray scale images of faces. The task was to categorize each face based on the emotion shown in the facial expression into one of seven categories (0=Angry, 1=Disgust, 2=Fear, 3=Happy, 4=Sad, 5=Surprise, 6=Neutral). Data set contains two columns, “emotion” and “pixels”. The “emotion” column contains a numeric code ranging from 0 to 6, inclusive, for the emotion that is present in the image. The “pixels” column contains a string surrounded in quotes for each image. The contents of this string are space-separated pixel values in row major order. test.csv contains only the “pixels” column and your task is to predict the emotion column.

4.6.2 Importing necessary libraries and packages

The image shows a Jupyter Notebook interface. At the top, the Jupyter logo is followed by the word "training" and a status message "Last Checkpoint: an hour ago (autosaved)". Below this is a menu bar with options: File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. Under the menu bar is a toolbar with icons for saving, adding, undo, redo, copy, paste, and navigation arrows, along with a "Run" button and a dropdown menu currently set to "Code". The main area of the notebook displays a code cell with the following Python code:

```
In [7]: import numpy as np
import cv2
from tensorflow.keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D
from keras.optimizers import Adam
from keras.layers import MaxPooling2D
from keras.preprocessing.image import ImageDataGenerator
```

Fig 4.8 Imported libraries and packages

4.6.3 Initialize training and validation generator

The pixels have been scaled between -1 and 1. The dataset has been split into train

and validation sets.

```
In [8]: train_dir = 'data/train'
        val_dir = 'data/test'
        train_datagen = ImageDataGenerator(rescale=1./255)
        val_datagen = ImageDataGenerator(rescale=1./255)
```

```
In [11]: train_generator = train_datagen.flow_from_directory(
        train_dir,
        target_size=(48,48),
        batch_size=64,
        color_mode="grayscale",
        class_mode='categorical')

        validation_generator = val_datagen.flow_from_directory(
        val_dir,
        target_size=(48,48),
        batch_size=64,
        color_mode="grayscale",
        class_mode='categorical')
```

```
Found 28709 images belonging to 7 classes.
Found 7178 images belonging to 7 classes.
```

Fig 4.9 Training and validation generator

4.6.4 Build the model using Convolutional Neural Networks(CNN)

The convolutional neural network, or CNN for short, is a specialized type of neural network model designed for working with two-dimensional image data, although they can be used with one-dimensional and three-dimensional data. Central to the convolutional neural network is the convolutional layer that gives the network its name. This layer performs an operation called a *convolution*. Here keras has been used with tensorflow as back-end for building Neural Networks. The layers to be added are:

- Convolution layer
- Pooling layer
- Batch normalization

- Activation Layer
- Dropout Layer
- Flatten Layer
- Dense layer

Convolution is a linear operation that involves the multiplication of a set of weights with the input, much like a traditional neural network. Given that the technique was designed for two-dimensional input, the multiplication is performed between an array of input data and a two-dimensional array of weights, called a filter or a kernel.

Batch normalization allows each layer of a network to learn by itself a little bit more independently of other layers. In a neural network, the activation function is responsible for transforming the summed weighted input from the node into the activation of the node or output for that input.

The Rectified linear activation(ReLU) function is a piecewise linear function that will output the input directly if it is positive, otherwise, it will output zero. It has become the default activation function for many types of neural networks because a model that uses it is easier to train and often achieves better performance.

A pooling layer is another building block of a CNN. Its function is to progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network. Pooling layer operates on each feature map independently.

In Data Augmentation we take each batch and apply some series of random transformations (random rotation, resizing, shearing) to increase generalizability of the model.

4.6.5 Compile and train the model

Now compile the model with any optimizer and any loss. I have used Adam optimizer and categorical_crossentropy loss. Now fit the model with some batch size and number of epochs and some parameters that make the model more efficient.

The feature will be extracted through the max-pooling method by creating the model with .h5 extension and then compile the model with loss and optimizer. Here we import

haar cascade for face recognition which is in XML format.

```
448/448 [=====] - 140s 313ms/step - loss: 0.4455 - accuracy: 0.8375 - val_loss: 1.1545 - val_accuracy: 0.6258
Epoch 45/50
448/448 [=====] - 141s 314ms/step - loss: 0.4340 - accuracy: 0.8447 - val_loss: 1.1865 - val_accuracy: 0.6246
Epoch 46/50
448/448 [=====] - 137s 307ms/step - loss: 0.4141 - accuracy: 0.8496 - val_loss: 1.1780 - val_accuracy: 0.6235
Epoch 47/50
448/448 [=====] - 138s 308ms/step - loss: 0.4008 - accuracy: 0.8560 - val_loss: 1.1810 - val_accuracy: 0.6211
Epoch 48/50
448/448 [=====] - 143s 319ms/step - loss: 0.3955 - accuracy: 0.8592 - val_loss: 1.2174 - val_accuracy: 0.6212
Epoch 49/50
448/448 [=====] - 139s 311ms/step - loss: 0.3878 - accuracy: 0.8575 - val_loss: 1.2282 - val_accuracy: 0.6207
Epoch 50/50
448/448 [=====] - 142s 317ms/step - loss: 0.3695 - accuracy: 0.8657 - val_loss: 1.1951 - val_accuracy: 0.6246
```

Fig 4.10 (a) Trial 1 : Epoch=50, accuracy=86.57%, loss = 36%

```
448/448 [=====] - 143s 318ms/step - loss: 0.1487 - accuracy: 0.9487 - val_loss: 1.4950 - val_accuracy: 0.6270
Epoch 95/100
448/448 [=====] - 143s 319ms/step - loss: 0.1478 - accuracy: 0.9471 - val_loss: 1.5222 - val_accuracy: 0.6321
Epoch 96/100
448/448 [=====] - 141s 315ms/step - loss: 0.1426 - accuracy: 0.9521 - val_loss: 1.5024 - val_accuracy: 0.6253
Epoch 97/100
448/448 [=====] - 141s 316ms/step - loss: 0.1508 - accuracy: 0.9460 - val_loss: 1.5296 - val_accuracy: 0.6325
Epoch 98/100
448/448 [=====] - 142s 316ms/step - loss: 0.1418 - accuracy: 0.9496 - val_loss: 1.5745 - val_accuracy: 0.6281
Epoch 99/100
448/448 [=====] - 143s 318ms/step - loss: 0.1393 - accuracy: 0.9508 - val_loss: 1.5273 - val_accuracy: 0.6295
Epoch 100/100
448/448 [=====] - 142s 318ms/step - loss: 0.1400 - accuracy: 0.9534 - val_loss: 1.5770 - val_accuracy: 0.6295
```

Fig 4.10 (b) Trial 2 : Epoch=100, accuracy=95.34%,loss=4%

4.6.6 Callback Functions

Callback functions are those functions which are called after every epoch during the training process. We will be using the following callback functions:

1. Reduce LR On Plateau: Training a neural network can plateau at times and we stop seeing any progress during this stage. Therefore, this function monitors the validation loss for signs of a plateau and then alters the learning rate by the

specified factor if a plateau is detected.

```
LR_reducer = Reduce LR On Plateau(monitor='val_loss', factor=0.9,  
                                  patience=3)
```

2. Early Stopping: At times, the progress stalls while training a neural network and we stop seeing any improvement in the validation accuracy (in this case). Majority of the time, this means that the network won't converge any further and there is no point in continuing the training process. This function waits for a specified number of epochs and terminates the training if no change in the parameter is found.

```
Early_stopper = EarlyStopping(monitor='val_acc',in_delta=0, patience=6,  
                              mode='auto')
```

3. Model Checkpoint: Training neural networks generally take a lot of time and anything can happen during this period that may result in loss of all the variables and weights. Creating checkpoints is a good habit as it saves your model after

every epoch. In case your training stops you can load the checkpoint and resume the process.

4.6.7 Using openCV haarcascade xml detect the bounding boxes of face in the webcam and predict the emotions

Takes pictures or webcam video as input. It detects all faces in each frame, and then classifies which emotion each face is expressing. Recognized emotions: Neutral, Angry, Sad, Happy, Disgust, Fear, Surprise. Training accuracy was 95% with the following requirements:

- Facial expression must be strong / exaggerated.
- Adequate Lighting (no shadows on face)
- Camera is at eye level or slightly above eye level



Fig 4.11 Detected the Facial Emotions

4.6.8 Testing the Model

The project started off by defining a loading mechanism and loading the images. Then a training set and a testing set are created. After this a fine model and a few callback functions are defined. The basic components of a convolutional neural network are considered and then training is done to the network. Create a folder named emojis and save the emojis corresponding to each of the seven emotions in the dataset.

4.7 APPLICATIONS

At Airports to observe the pilot's psychological condition before take-off. **At Hospitals** can be performed on a psychological disorder patient by a psychiatric doctor.

By the Crime Department used as a lie detector. **Social websites** feedback depicted through the face in the absence of written feedback or rating.

Social Welfare gathering information would be profitable in the case of deaf and dumb people (autism patients). **Driver Monitoring** monitoring driver facial expressions while driving. **OTT Platforms and Video Trailers** monitor the viewers emotions while watching movies or video trailers and generate the feedback

Showrooms and Shops after purchasing any product, the buyer can give their feedback regarding the service provided to them.

Text messages communication through emojis is way too easier than typing long messages. Robust spontaneous expression recognizers can be developed and deployed in real-time systems and used in building emotion sensitive HCI interfaces. The project can have an impact on our day to day life by enhancing the way we interact with computers or in general, our surrounding living and work spaces. High correct recognition rate , significant performance improvements in our system. Promising results are obtained under face registration errors, fast processing time.

System is fully automatic and has the capability to work with video feeds as well as images. It is able to recognize spontaneous expressions. Our system can be used in Digital Cameras where the image is captured only when a person smiles, or if the person doesn't blink his eyes. In security systems which can identify a person, in any form of expression he presents himself. Rooms in homes can set the lights, television to a person's taste when they enter the room.

This can be used by the doctors in understanding the intensity of pain or illness of a deaf patient.

CHAPTER 5

RESULTS AND DISCUSSION

In the above figures, i.e., output is the window where the user expressions are captured by the webcam and the respective emotions are detected. On detecting the emotion, the respective emoticon is shown on the left side of the screen. This emoticon changes with the change in the expression of the person in front of the webcam. Hence changes with the change in the expression of the person in front of the webcam. Hence this real time application is very beneficial in various fields like psychology, computer science, linguistics, neuroscience and related disciplines.

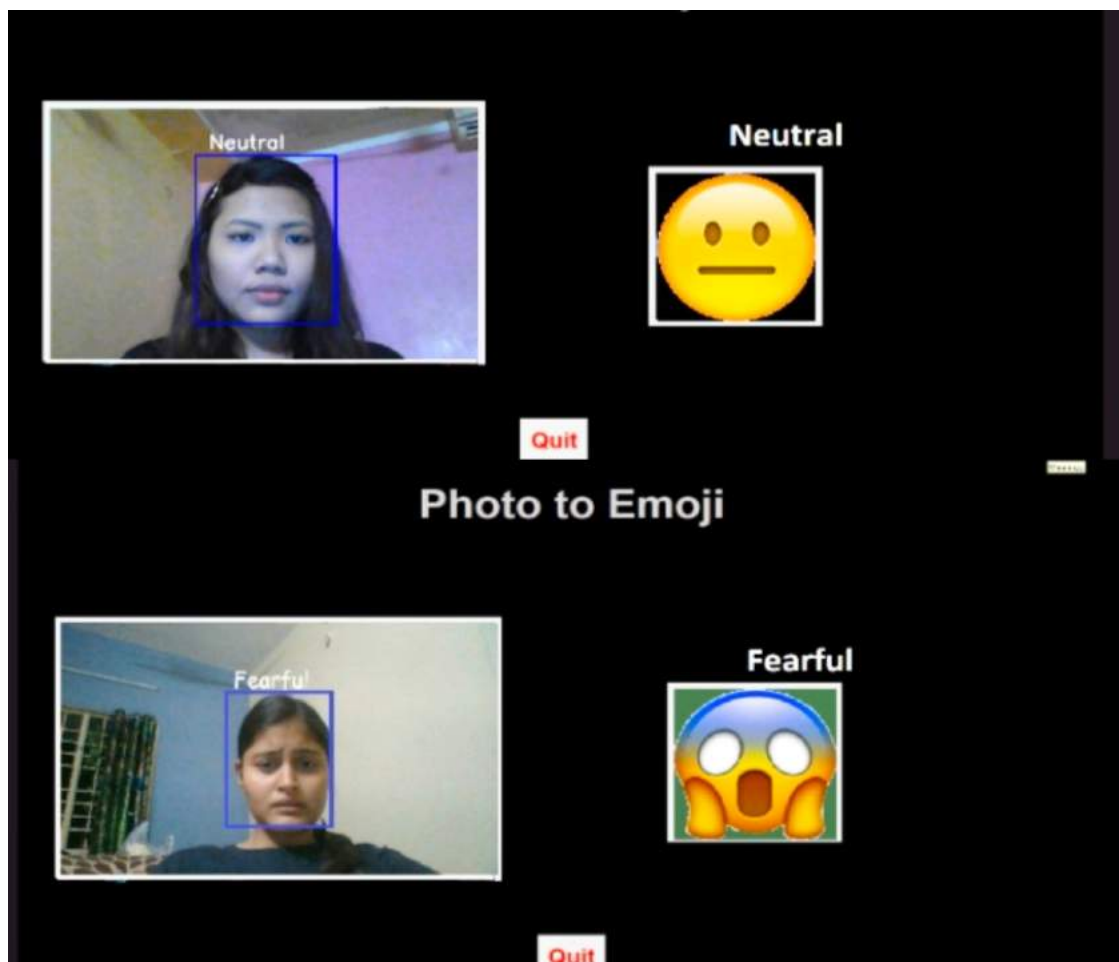


Fig 5.1(a) Emoji Generated of the emotions i)Neutral ii) Fearful

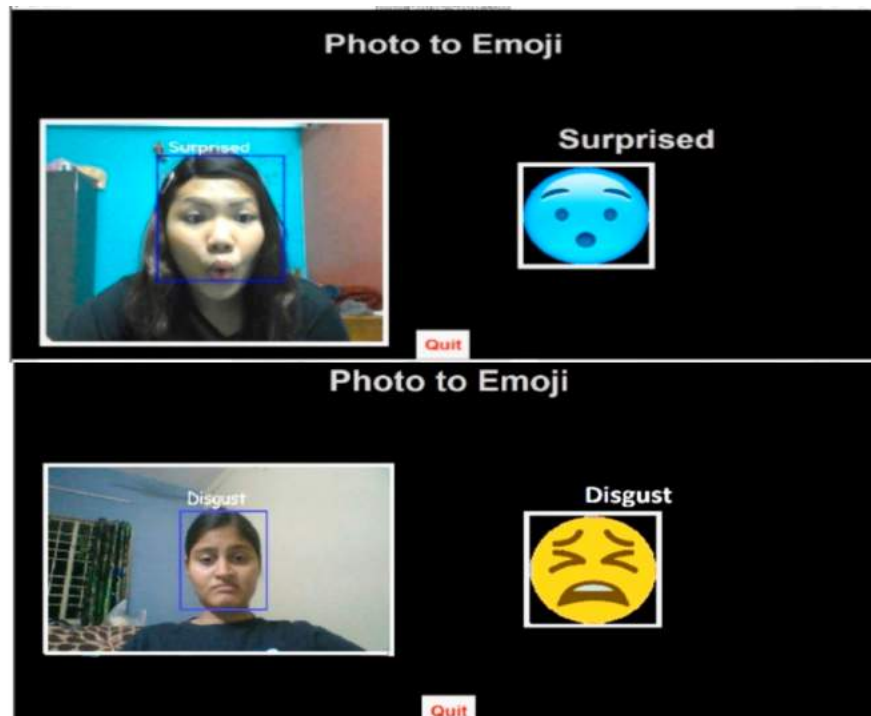


Fig 5.1(b) Emoji Generated of the emotions i) Surprised ii) Disgust

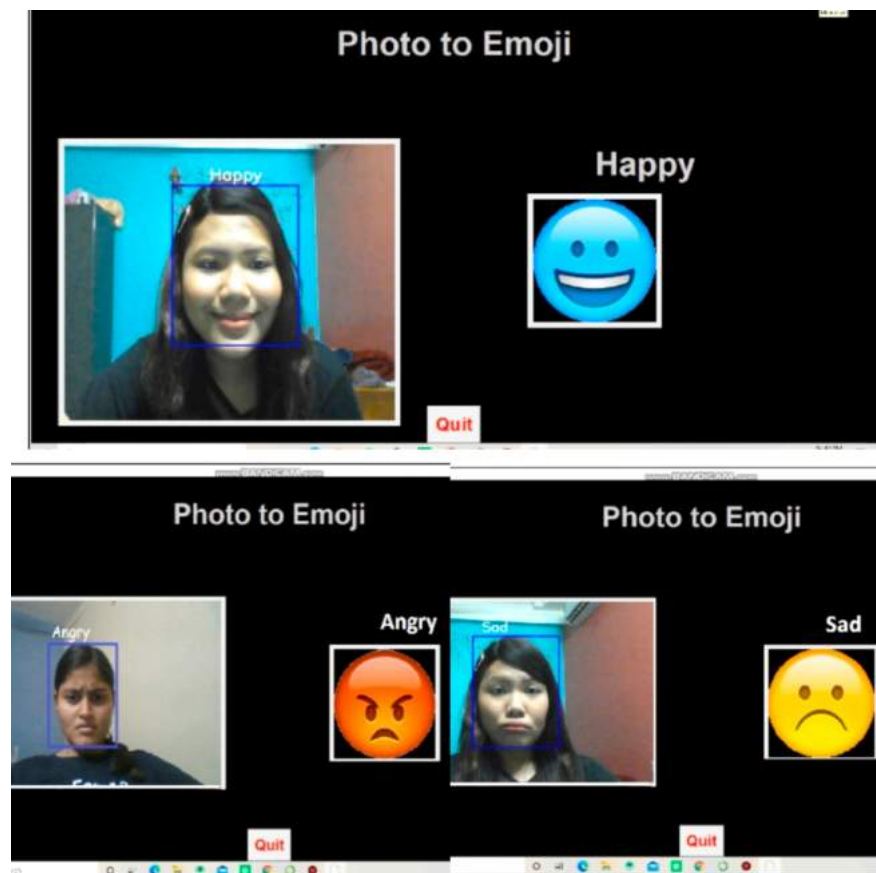


Fig 5.1(c) Emoji Generated of the emotions i) Happy ii) Angry iii) Sad

Results were obtained by experimenting with the CNN algorithm. It was observed that the loss over training and test set decreased with each epoch. The batch size was 256, which was kept constant over all experiments.

The following changes were made in the neural network architecture to achieve good results:

- Number of epochs: It was observed that the accuracy of the model increased with increasing number of epochs. However, a high number of epochs resulted in overfitting. It was concluded that eight epochs resulted in minimum overfitting and high accuracy.
- Number of layers: The neural network architecture consists of three hidden layers and a single fully connected layer. A total of six convolution layers were built, using 'relu' as the activation function.
- Filters: The neural network accuracy on the dataset varied on the number of filters applied to the image. The number of filters for the first two layers of the network was 64, and it was kept 128 for the third layers of the network.
- Accuracy: The final, state-of-the-art-model gave a training accuracy of 79.89% and a test accuracy 60.12% as shown in the table. The architecture used could correctly classify 22936 out of 28709 images from the train set and 2158 out of 3589 images from the test set.

Table 5.1 Accuracy obtained over the three experiments

Experiment	Training Accuracy	Test Accuracy	Validation Accuracy
Experiment 1	62.46	58.03	86.57
Experiment 2	79.89	60.12	95.39

Table 5.2 Comparison of the proposed system with other related works:

Related work	Algorithm	Dataset	Results
Kumar, Kumar, & Sanyal, 2016	CNN	FERC-2013	Around 90%
Amin, Chase & Sinha, 2017	CNN	FER-2013	60.37
Shan, Guo, You, Lu & Bie, 2017	KNN	JAFFE, CK+	65.11, 77.27
Kulkarni, Bagal, 2015	Gabor, Log Gabor	FACES	82%, 87%
Minaee, & Abdolrashidi, 2019	Attentional CNN	FER2013	70.02%
Proposed	CNN	FER2013	95.39%

Loss and accuracy over time

It can be observed that the loss decreases, and the accuracy increases with each epoch. The training versus testing curve for accuracy remains ideal over the first five epochs, after which it begins to deviate from the ideal values. The training and test accuracy along with the training and validation loss obtained for the FER2013 dataset using CNN are given in Table.5.3.

Table 5.3 Accuracy per epoch

Epoch	Training Accuracy	Validation Accuracy
1	29.10	43.33
2	47.81	50.65
3	55.60	56.90
4	60.13	57.65
5	64.07	57.95
6	67.00	59.63
7	69.95	59.01
8	72.88	60.13
50	86.57	62.46
100	95.39	64.95

Confusion Matrix

The confusion matrix generated over the test data is shown in figure 7. The dark blocks along the diagonal show that the test data has been classified well. It can be observed that the number of correct classifications is low for disgust, followed by fear. The numbers on either side of the diagonal represent the number of wrongly classified images. As these numbers are lower compared to the numbers on the diagonal, it can be concluded that the algorithm has worked correctly and achieved state of the art results.

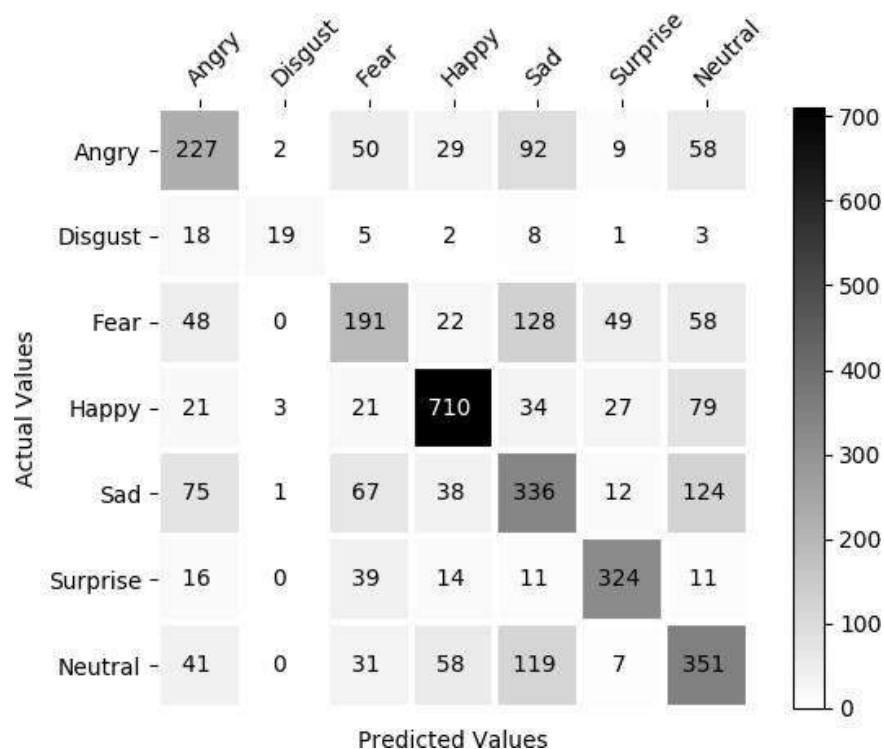


Fig 5.2. Confusion matrix

CHAPTER 6

CONCLUSION AND FUTURE WORK

CONCLUSION

In this paper, an approach for FER using CNN has been discussed. A CNN model on the FER2013 dataset was created and experiments with the architecture were conducted to achieve a test accuracy of 0.6212 and a validation accuracy of 0.9508. This state-of-the-art model has been used for classifying emotions of users in real time using a webcam. The webcam captures a sequence of images and uses the model to classify emotions and generate the corresponding emoji. Proposed is a human emotion detector using emoticons using machine learning, python to predict emotions of the people and represent them using emoticons. These include image acquisition, preprocessing of an image, face detection, feature extraction, classification and then when the emotions are classified the system assigns the user particular music according to his emotion. The main aim of this project is to develop an automatic facial emotion recognition system in which an emoticon is used for giving the output for individuals thus assigning them various therapies or solutions to relieve them from stress. The emotions used for the experiments include happiness, Sadness, Surprise, Fear, Disgust, and Anger that are universally accepted.

FUTURE WORK

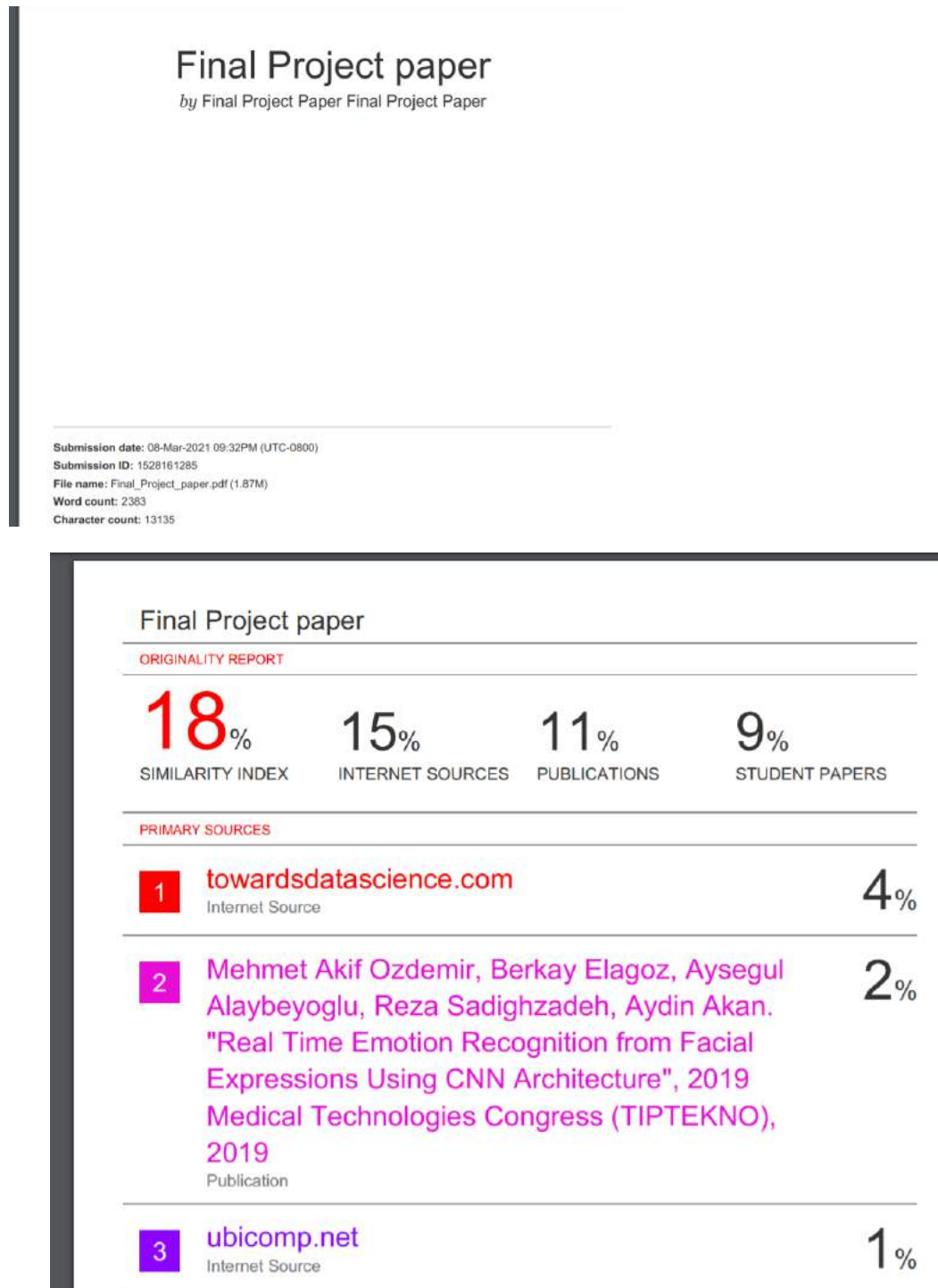
Face expression recognition systems have improved a lot over the past decade. The focus has definitely shifted from posed expression recognition to spontaneous expression recognition. Promising results can be obtained under face registration errors, fast processing time and significant performance improvements can be obtained in our system. System is fully automatic and has the capability to work with images feed. It is able to recognize spontaneous expressions. The system can be used in Digital Cameras wherein the image can be captured only when the person smiles. In security systems which can identify a person, in any form of expression he presents himself. Doctors can use the system to understand the intensity of pain or illness of a deaf patient. Our system can be used to detect and track a user's state of mind, and in mini-marts, shopping centers to view the feedback of the customers to enhance the business etc.

REFERENCES :

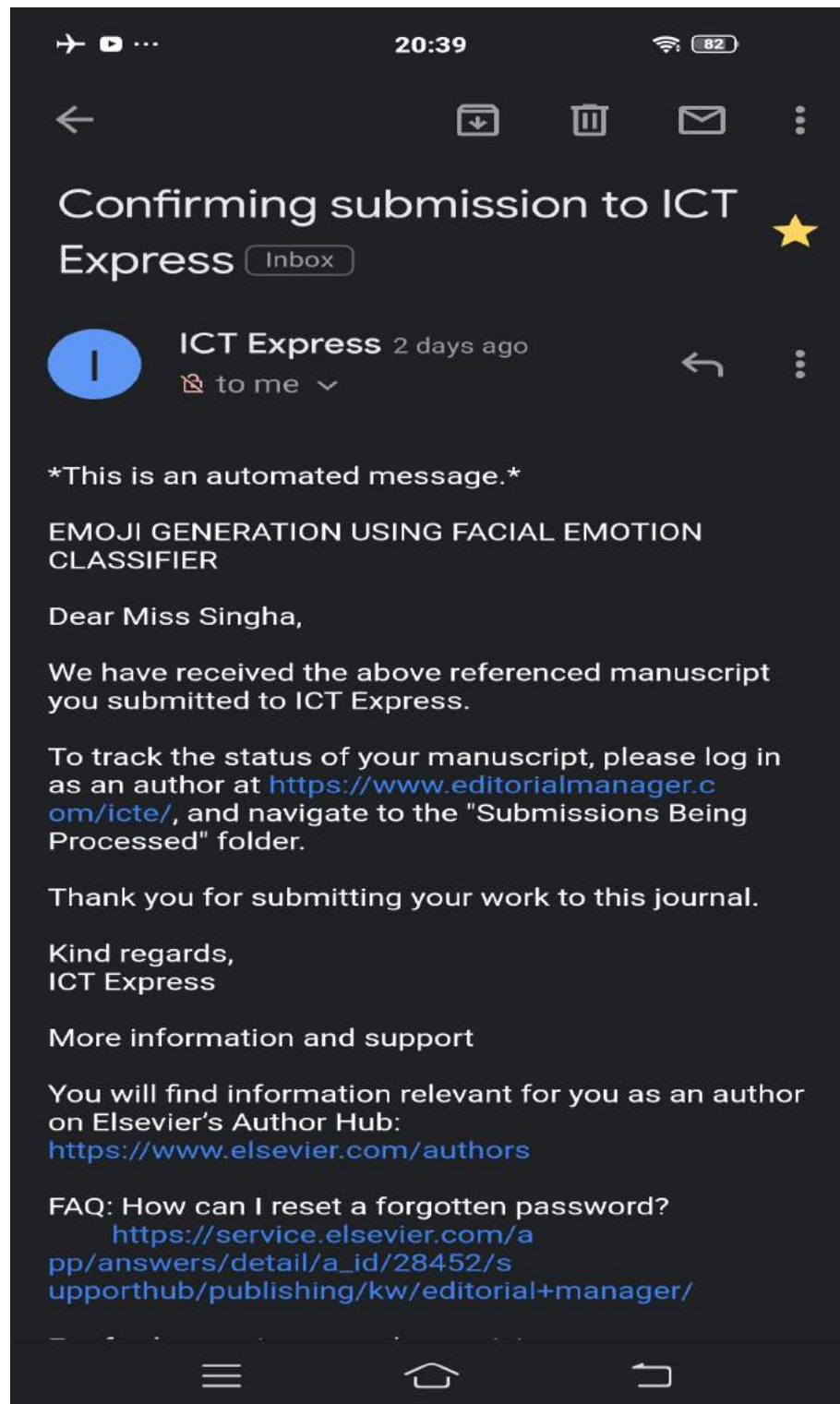
- ❑ H.-D. Nguyen, S. Yeom, G.-S. Lee, H.-J. Yang, I. Na, and S. H. Kim, "Facial Emotion Recognition Using an Ensemble of MultiLevel Convolutional Neural Networks," *International Journal of Pattern Recognition and Artificial Intelligence*, 2018.
- ❑ T. Cao and M. Li, "Facial Expression Recognition Algorithm Based on the Combination of CNN and K-Means," presented at the *Proceedings of the 2019 11th International Conference on Machine Learning and Computing*, Zhuhai, China, 2019.
- ❑ N. Christou and N. Kanojiya, "Human Facial Expression Recognition with Convolutional Neural Networks," Singapore, 2019, pp. 539-545: Springer Singapore
- ❑ A. Sajjanhar, Z. Wu, and Q. Wen, "Deep learning models for facial expression recognition," in *2018 Digital Image Computing: Techniques and Applications (DICTA)*, 2018, pp. 1-6: IEEE.
- ❑ J. Chen, Y. Lv, R. Xu, and C. Xu, "Automatic social signal analysis: Facial expression recognition using difference convolution neural network," *Journal of Parallel and Distributed Computing*, vol. 131, pp. 97-102, 2019.
- ❑ Al-Sumaidae, Saadoon AM, et al, Multi-gradient features and elongated quinary pattern encoding for image-based facial expression recognition, *Pattern Recognition*, 2017, pp. 249—263.
- ❑ Barsoum, Emad, et al, Training deep networks for facial expression recognition with crowd-sourced label distribution, *ACM International Conference on Multimodal Interaction ACM*, 2016, pp. 279—283.
- ❑ Martinez, Brais, et al, Automatic analysis of facial actions: A survey, *IEEE Transactions on Affective Computing*, 2017.
- ❑ J. M. B. Fugate, A. J. O'Hare, and W. S. Emmanuel, "Emotion words: Facing change," *Journal of Experimental Social Psychology*, vol. 79, pp. 264-274, 2018.
- ❑ K. Clawson, L. Delicato, and C. Bowerman, "Human Centric Facial Expression Recognition," 2018

APPENDIX

(A) PLAGIARISM REPORT



(B) PAPER SUBMISSION MAIL



EMOJI GENERATION USING FACIAL EMOTION CLASSIFIER

Daspakhla, M. Shweta Singh and Dr. A. C. Santha Shela
Sathyabama Institute of Science and Technology, Chennai, India
E-Mail: d.pakhla259@gmail.com

ABSTRACT

Face expressions are an intrinsic aspect of nonverbal communication and play a significant role in Human-Computer Interaction. The formation of facial emotions is a human-computer interaction device. Emoji generation in real time based on a person's facial expression has always been difficult. Human social conversations depend on facial expressions. Since the world is becoming more technologically sophisticated day in day out, there are more interactive encounters, such as text messages, than physical ones. Emotions promote virtual social interaction by reducing the amount of words exchanged. This paper describes an Emoji generation methodology based on Facial Expression Recognition (FER) and Convolutional Neural Networks (CNN) coupled with Machine Learning and Deep Learning. This CNN-based model can be put to work to evaluate feelings as users watch movie trailers or video lessons, as well as to help people with autism regulate their emotions.

Keywords: Emotions, Human-Computer Interaction, Facial Expression Recognition, Machine Learning, Deep learning.

1. INTRODUCTION

Emojis and avatars are visual representations of nonverbal signals. These signals have been ingrained in online chatting, product reviews, brand emotion, and a variety of other activities. It also encourages future data science exploration into emoji-driven storytelling.

Human-generated product online reviews



FIG(1) Classification of 7 basic emotions

We construct a deep neural classifier to identify facial expression and generate emotions for this. We use the Haar cascade classifier for face detection before using the deep neural classifier, and then the neural classifier for expression classification. Facial expression recognition employs the deep CNN process, as well as the Haar Cascade face classifier and computer vision techniques. To remove higher-level functions, CNN convolve with the original images or function maps using a convolutional kernel.

A. SYSTEM ARCHITECTURE



FIG (2) System Architecture

The system architecture is as follows: our model uses the camera with the help of an open-source computer vision library; it is a library of programming functions mainly aimed at real-time computer vision and detects faces and classifies the expression on faces. For each classified expression there will be a generated emotion as the output in the application.

need for human intervention, which automatically detects the important features without any human supervision.

The aim of this deep learning project is to effectively identify human facial expressions so that corresponding emoji or avatars can be filtered and mapped. The application can be used in large corporations or businesses to receive real-time consumer reviews. The

Module 1: Data Pre-processing

Let's talk about the datasets until we get into the preprocessing operations. The datasets have a significant impact on model efficiency.

FER2013 is the name of the datasets we used in our article, and it is available on Kaggle. It contains 35,685 grayscale images of faces with a resolution of 48x48 pixels. Happy, neutral, sad, anger, disappointment, disgust, and fear are the emotions displayed in the facial expressions that are classified as happy, neutral, sad, angry, surprise, disgust, and fear.

Data visualization is one of the preprocessing measures that aids in envisioning the total number of images. Another stage in the preprocessing is data augmentation, which involves rotating and flipping each image in our datasets. More photographs are added to our datasets as a result of this. It is being achieved in order to broaden the range of data that can be used to train models. After that, we normalize the data before saving and targeting it.

Module 2: Training the CNN

Before we undergo training, we must first build the CNN, which consists of several layers such as Conv2D, MaxPooling2D, Flatten, Dropout, and Dense.

The Keras library's sequential API helps us to incrementally build a new layer for our model. The Conv2D is the initial layer, with 200 filters and a filter size or kernel size

technology (FERET) dataset, Paul Ekman [2] has presented 7 universal expressions (anger, fearful, happy, sad, neutral, disgust, and surprise) with the positioning of faces, and the muscular movements required to create these expressions in his study (Ekman, 1997). The Facial Action Coding System (FACS), developed by Swedish anatomist Carl-Herman Hjortskov, is a coding system used to taxonomize human facial movements based on their appearance on the face. This system, which was later adopted by Ekman & Friesen (2003), is also a useful method of classifying human expressions. FER systems were mostly implemented using the FACS in the past. However, recently there has been a trend to implement FER using classification algorithms such as SVM, neural networks, and the Fisherface algorithm (Alshamsi, Kepuska & Meng, 2017 [1], Fathallah, Abdi & Douik, 2017 [3], Lyons, Budynek & Akamatsu, 1999 [7]).

There are several datasets available for research in the field of Facial Expression Recognition, such as the Japanese Female Facial Expressions (JAFFE), Extended Cohn Kanade dataset (CK+), and the FER2013 dataset (Cohn, Cohn & Tian, 2000 [5]; Lucy et al., 2010 [6]; Goodfellow et al., 2013 [4]). The type and number of images, the method of labelling the images varies in each dataset. The CK+ dataset uses the FACS system for labelling faces and contains the Action Units (AU's) for each facial image. There are several challenges with implementing the FER system. Most datasets

optimization algorithm. This necessitates the selection of an error function, commonly termed as a loss function, to estimate the model's loss and modify the weights to minimize the loss on the next assessment.

In multi-class classification tasks, categorical cross-entropy also called Softmax Loss is a loss function. These are tasks in which an example can only fit into one of several possible categories, and the model must choose one. The categorical cross-entropy loss function computes an example's loss by summing the following values:

$$Loss = - \sum_{i=1}^n y_i \cdot \log(\hat{y}_i)$$

where y_i is the corresponding target value, $\hat{y}_i$ is the i -th scalar value in the model output, and output size is the number of scalar values in the model output. This loss is an excellent indicator of how distinct two discrete probability distributions are from one another. We have seven categories: 'Angry', 'Disgusted', 'Fearful', 'Happy', 'Sad', 'Neutral', and 'Surprised', so we use Categorical cross entropy as the loss function and Adam as the optimizer for stochastic gradient descent on our model.

Finally, the CNN model is trained for 100 epochs with seven classes, on detecting the class of images with different facial expression which will be then converted into emotions.

labels in the datasets are broadly classified, which means that in real time there might be some expressions which the system might be able to classify correctly. Many FER systems, such as Affective, and Microsoft's Emotion API (McDuff et al., 2016; Lim, 2015 [9]). These systems have become very popular in applications where FER is required.

3. PROPOSED SYSTEM

Beyond laboratory experiments, one of the main topics in facial recognition is human emotion expression, which can produce both technical and everyday applications. This project creates a deep learning model that can classify a given picture of a human facial expression into one of the seven essential human emotions. The classified emotion would then be mapped to an emoji or avatar.



FIG(3) The emoji are correctly mapped on face corresponding faces. The input will be the image of the expression, and output will be shown as above

The main aim of this project is to create a model that can identify seven basic emotions: happy, sad, surprise, angry, disgust, neutral, and fear, and to improve on the baseline accuracy. Furthermore, our project aims to analyze the outcomes of our model in terms of accuracy for each class. The model is expected to perform well emotion recognition in the future, with more complex state variance than lab condition photos.

of OpenCV is designed to detect the frontal face by training thousands of images.

We label seven probabilities, '0-6' for different face expressions, and also setting the boundary rectangle color using the RGB value blue which shows the detected face. In our last step, we use the OpenCV library to run an infinite loop to use our web camera in which we detect the face using the Cascade Classifier. The code denotes the usage of webcam. Based on which probability is higher, the label will be chosen and displayed above the detected faces in white fonts.

4. RESULT AND DISCUSSION

The model was executed in an Anaconda environment by using Python Language. It uses open-source libraries like pandas, NumPy, and classification algorithms: TensorFlow, Keras, etc.

The experimental setup was done in Intel (5-7200U) CPU @2.50GHz with 8GB RAM

For 100 epochs, the model was conditioned. However, unless over-fitting happens, we will train for further epochs to improve precision. We will see that our model has a 95.3 percent accuracy after the 100th epoch. This indicates that it has been well trained and is not over-fitted.

facial expressions through the webcam and also by using images as input data.

6. REFERENCES

- [1] Alshamsi, H., Kepuska, V., & Meng, H. (2017, October). Real time automated facial expression recognition app development on smartphones. In 2017 8th IEEE Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON) (pp.384-392). IEEE.
- [2] Ekman, P. (1994). Strong evidence for universals in facial expressions: a reply to Russell & 39's mistaken critique. Ekman, R. (1997). What the face reveals: Basic and applied studies of spontaneous expression using the Facial Action Coding System (FACS). Oxford University Press, USA Ekman, P., & Friesen, W. V. (2003). Unmasking the face: A guide to recognizing emotions from facial clues. Ishk.
- [3] Fathallah, A., Abdi, L., & Douik, A. (2017, October). Facial expression recognition via deep learning. In 2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA) (pp. 745-750). IEEE.
- [4] Goodfellow, I. J., Erhan, D., Carrier, P. L., Courville, A., Mirza, M., Hamner, B., ... & Zhou, Y. (2013, November). Challenges in representation learning: A report on three machine learning contests. In

This paper demonstrates an improved FER model with 95 percent accuracy, and the produced outcome correlates to the person's facial expression. It demonstrates how the model can work in real-time by capturing facial expressions through the webcam and also by using images as input data.

6. REFERENCES

- [2] Kanade, T., Cohn, J. F., & Tian, Y. (2000). Comprehensive database for facial expression analysis. In Proceedings Fourth IEEE International Conference on Automatic Face and Gesture Recognition (Cat. No. PR000280) (pp. 46-53). IEEE.
- [6] Lucy, P., Cohn, J. F., Kanade, T., Sengco, J., Ambadar, Z., & Matthews, I. (2010, June). The extended cohn-kanade dataset (ck+): A complete dataset for action

(D) SOURCE CODE :

Test.py

```
import numpy as np

import cv2

from tensorflow.keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D
from keras.optimizers import Adam
from keras.layers import MaxPooling2D
from keras.preprocessing.image import ImageDataGenerator

train_dir = 'data/train'
val_dir = 'data/test'
train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen = ImageDataGenerator(rescale=1./255)
validation_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(48,48),
    batch_size=64,
    color_mode="grayscale",
    class_mode='categorical')

emotion_model = Sequential()
emotion_model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape =
(48,48,1)))
emotion_model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
emotion_model.add(Dropout(0.25))
emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='relu'))
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='relu'))
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
emotion_model.add(Dropout(0.25))
emotion_model.add(Flatten())
```



```

emotion_model.add(Dense(1024, activation='relu'))
emotion_model.add(Dropout(0.5))
emotion_model.add(Dense(7, activation='softmax'))
cv2ocl.setUseOpenCL(False)
emotion_dict = {0: "Angry", 1: "Disgusted", 2: "Fearful", 3: "Happy", 4: "Neutral", 5:
"Sad", 6: "Surprised"}
emotion_model.compile(loss='categorical_crossentropy',optimizer=Adam(lr=0.0001,
decay=1e-6),metrics=['accuracy'])
emotion_model_info = emotion_model.fit_generator(
    train_generator,
    steps_per_epoch=28709 // 64,
    epochs=100,
    validation_data=validation_generator,
    validation_steps=7178 // 64)
emotion_model.save_weights('emotion_model.h5')
cap = cv2.VideoCapture(0)
while True:
# Find haar cascade to draw bounding box around face
ret, frame = cap.read()
    if not ret:
        break
    bounding_box=cv2.CascadeClassifier(r'C:\Users\Lenovo\Downloads\opencv-master\op
encv-master\data\haarcascades\haarcascade_frontalface_default.xml')
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    num_faces      =      bounding_box.detectMultiScale(gray_frame,scaleFactor=1.3,
minNeighbors=5)
    for (x, y, w, h) in num_faces:
        cv2.rectangle(frame, (x, y-50), (x+w, y+h+10), (255, 0, 0), 2)
        roi_gray_frame = gray_frame[y:y + h, x:x + w]
        cropped_img  =  np.expand_dims(np.expand_dims(cv2.resize(roi_gray_frame, (48,
48)), -1), 0)
        emotion_prediction = emotion_model.predict(cropped_img)
        maxindex = int(np.argmax(emotion_prediction))
cv2.putText(frame,          emotion_dict[maxindex],          (x+20,          y-60),

```

```
cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2, cv2.LINE_AA)
```

```
cv2.imshow('Video', cv2.resize(frame,(1200,860),interpolation = cv2.INTER_CUBIC))
```

```
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
        break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Gui.py

```
import tkinter as tk
```

```
from tkinter import *
```

```
import cv2
```

```
from PIL import Image, ImageTk
```

```
import os
```

```
import numpy as np
```

```
import numpy as np
```

```
import cv2
```

```
from keras.models import Sequential
```

```
from keras.layers import Dense, Dropout, Flatten
```

```
from keras.layers import Conv2D
```

```
from keras.optimizers import Adam
```

```
from keras.layers import MaxPooling2D
```

```
from keras.preprocessing.image import ImageDataGenerator
```

```
emotion_model = Sequential()
```

```
emotion_model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape  
=(48,48,1)))
```

```
emotion_model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
```

```
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
emotion_model.add(Dropout(0.25))
```

```
emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='relu'))
```

```
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='relu'))
```

```
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
emotion_model.add(Dropout(0.25))
```

```

emotion_model.add(Flatten())
emotion_model.add(Dense(1024, activation='relu'))
emotion_model.add(Dropout(0.5))
emotion_model.add(Dense(7, activation='softmax'))
emotion_model.load_weights('emotion_model.h5')
cv2ocl.setUseOpenCL(False)
emotion_dict = {0: "   Angry   ", 1: "Disgusted", 2: "   Fearful   ", 3: "   Happy   ", 4: "
Neutral   ", 5: "   Sad   ", 6: "Surprised"}
emoji_dist={0:"C:\\Users\\Lenovo\\Desktop\\emoji-creator-project-code\\emojis\\Angry.p
ng",1:"C:\\Users\\Lenovo\\Desktop\\emoji-creator-project-code\\emojis\\Disgusted.png",
2:"C:\\Users\\Lenovo\\Desktop\\emoji-creator-project-code\\emojis\\Fearful.png",3:"C:\\
Users\\Lenovo\\Desktop\\emoji-creator-project-code\\emojis\\Happy.png",4:"C:\\Users\\
Lenovo\\Desktop\\emoji-creator-project-code\\emojis\\Neutral.png",5:"C:\\Users\\Lenov
o\\Desktop\\emoji-creator-project-code\\emojis\\Sad.png",6:"C:\\Users\\Lenovo\\Desкто
p\\emoji-creator-project-code\\emojis\\Surprise.png"}
global last_frame1
last_frame1 = np.zeros((480, 640, 3), dtype=np.uint8)
global cap1
show_text=[0]
def show_vid():
    cap1 = cv2.VideoCapture(0)
    if not cap1.isOpened():
        print("can't open the camera1")
    flag1, frame1 = cap1.read()
    frame1 = cv2.resize(frame1,(600,500))
bounding_box=cv2.CascadeClassifier(r'C:\\Users\\Lenovo\\Downloads\\opencv-master\\op
encv-master\\data\\haarcascades\\haarcascade_frontalface_default.xml')
    gray_frame = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
        num_faces  =  bounding_box.detectMultiScale(gray_frame,scaleFactor=1.3,
minNeighbors=5)
        for (x, y, w, h) in num_faces:
            cv2.rectangle(frame1, (x, y-50), (x+w, y+h+10), (255, 0, 0), 2)
            roi_gray_frame = gray_frame[y:y + h, x:x + w]
            cropped_img = np.expand_dims(np.expand_dims(cv2.resize(roi_gray_frame, (48,

```

```

48)), -1),0)
    prediction = emotion_model.predict(cropped_img)
    maxindex = int(np.argmax(prediction))
cv2.putText(frame1, emotion_dict[maxindex], (x+20, y-60),
cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2, cv2.LINE_AA)
    show_text[0]=maxindex
    if flag1 is None:
        print ("Major error!")
    elif flag1:
        global last_frame1
        last_frame1 = frame1.copy()
        pic = cv2.cvtColor(last_frame1, cv2.COLOR_BGR2RGB)
        img = Image.fromarray(pic)
        imgtk = ImageTk.PhotoImage(image=img)
        lmain.imgtk = imgtk
        lmain.configure(image=imgtk)
        lmain.after(10, show_vid)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        exit()
def show_vid2():
    frame2=cv2.imread(emoji_dist[show_text[0]])
    pic2=cv2.cvtColor(frame2,cv2.COLOR_BGR2RGB)
    img2=Image.fromarray(frame2)
    imgtk2=ImageTk.PhotoImage(image=img2)
    lmain2.imgtk2=imgtk2
    lmain3.configure(text=emotion_dict[show_text[0]],font=('arial',45,'bold'))
    lmain2.configure(image=imgtk2)
    lmain2.after(10, show_vid2)

if __name__ == '__main__':
    root=tk.Tk()

    #img =
    ImageTk.PhotoImage(Image.open("C:\\Users\\Lenovo\\Downloads\\logo.png"))
    heading = Label(root,bg='black')

```

```

    heading.pack()
heading2=Label(root,text="Photo to Emoji",pady=20,
font=('arial',45,'bold'),bg='black',fg='#CDCDCD')
    heading2.pack()
    lmain = tk.Label(master=root,padx=50,bd=10)
    lmain2 = tk.Label(master=root,bd=10)
lmain3=tk.Label(master=root,bd=10,fg="#CDCDCD",bg='black')
    lmain.pack(side=LEFT)
    lmain.place(x=50,y=250)
    lmain3.pack()
    lmain3.place(x=960,y=250)
    lmain2.pack(side=RIGHT)
    lmain2.place(x=900,y=350)
    root.title("Photo To Emoji")
    root.geometry("1400x900+100+10")
    root['bg']='black'
exitbutton=Button(root,text='Quit',fg="red",command=root.destroy,font=('arial',25,'bold')
).pack(side = BOTTOM)
    show_vid()
    show_vid2()
    root.mainloop()

```